Document number: P1350R0
Date: 2018-11-07
Project: Programming Language C++
Reference: ISO/IEC IS 14882:2017
Reply to: William M. Miller
Edison Design Group, Inc.
wmm@edg.com

Core Language Working Group "tentatively ready" Issues for the November, 2018 (San Diego) meeting

Section references in this document reflect the section numbering of document WG21 N4778.

1636. Bits required for negative enumerator values

Section: 9.6 [dcl.enum] **Status:** tentatively ready **Submitter:** Hyman Rosen **Date:** 2013-03-07

According to 9.6 [dcl.enum] paragraph 7,

> For an enumeration whose underlying type is fixed, the values of the enumeration are the values of the underlying type. Otherwise, for an enumeration where e_{min} is the smallest enumerator and e_{max} is the largest, the values of the enumeration are the values in the range b_{min} to b_{max}, defined as follows: Let K be 1 for a two's complement representation and 0 for a one's complement or sign-magnitude representation. b_{max} is the smallest value greater than or equal to max($|e_{min}|$-K,$|e_{max}|$) and equal to 2^M-1, where M is a non-negative integer. b_{min} is zero if e_{min} is non-negative and -(b_{max}+K) otherwise. The size of the smallest bit-field large enough to hold all the values of the enumeration type is max(M,1) if b_{min} is zero and M+1 otherwise.

The result of these calculations is that the number of bits required for

```
enum { N = -1, Z = 0 }
```

is 1, but the number required for

```
enum { N = -1 }
```

is 2. This is surprising. This could be fixed by changing $|e_{max}|$ to e_{max}.

Proposed resolution (June, 2018):

Change 9.6 [dcl.enum] paragraph 8 as follows:

> b_{max} is the smallest value greater than or equal to $max(|e_{\text{min}}| - K, ~~|e_{\text{max}}|~~)$ **, e_{max})** and equal to $2^M - 1$, where M is a non-negative integer.

1781. Converting from `nullptr_t` to `bool` in overload resolution

Section: 11.3.1.5 [over.match.conv] **Status:** tentatively ready **Submitter:** Hubert Tong **Date:** 2013-09-26

According to 11.3.1.5 [over.match.conv] paragraph 1, when a class type `S` is used as an initializer for an object of type `T`,

> The conversion functions of `S` and its base classes are considered. Those non-explicit conversion functions that are not hidden within `S` and yield type `T` or a type that can be converted to type `T` via a standard conversion sequence (11.3.3.1.1 [over.ics.scs]) are candidate functions.

Because conversion from `std::nullptr_t` to `bool` is only permitted in direct-initialization (7.3.13 [conv.fctptr]), it is not clear whether there is a standard conversion sequence from `std::nullptr_t` to `bool`, considering that an implicit conversion sequence is intended to model copy-initialization. Should 11.3.1.5 [over.match.conv] be understood to refer only to conversions permitted in copy-initialization, or should the form of the initialization be considered? For example,

```
struct SomeType {
  operator std::nullptr_t();
};
bool b{ SomeType() };     // Well-formed?
```

Note also 11.3.3.2 [over.ics.rank] paragraph 4, which may bear on the intent (or, alternatively, might describe a situation that cannot arise):

> A conversion that does not convert a pointer, a pointer to member, or `std::nullptr_t` to `bool` is better than one that does.

See also issues 2133 and 2243.)

Proposed resolution (June, 2018):

1. Change 7.3.14 [conv.bool] paragraph 1 as follows:

> A prvalue of arithmetic, unscoped enumeration, pointer, or pointer-to-member type can be converted to a prvalue of type `bool`. A zero value, null pointer value, or null member pointer value is converted to `false`; any other value is converted to `true`. ~~For direct-initialization (9.3 [dcl.init]), a prvalue of type `std::nullptr_t` can be converted to a prvalue of type `bool`; the resulting value is `false`.~~

2. Add the following bullet before 9.3 [dcl.init] bullet 17.8:

> The semantics of initializers are as follows. The *destination type* is the type of the object or reference being initialized and the *source type* is the type of the initializer expression. If the initializer is not a single (possibly parenthesized) expression, the source type is not defined.
>
> - ...
> - **Otherwise, if the initialization is direct-initialization, the source type is `std::nullptr_t`, and the destination type is `bool`, the initial value of the object being initialized is `false`.**
> - Otherwise, the initial value of the object being initialized is the (possibly converted) value...

3. Change 11.3.3.2 [over.ics.rank] bullet 4.1 as follows:

> Standard conversion sequences are ordered by their ranks: an Exact Match is a better conversion than a Promotion, which is a better conversion than a Conversion. Two conversion sequences with the same rank are indistinguishable unless one of the following rules applies:
>
> - A conversion that does not convert a pointer~~,~~ **or** a pointer to member~~, or `std::nullptr_t`~~ to `bool` is better than one that does.
> - ...

This resolution also resolves issue 2133.

2133. Converting `std::nullptr_t` to `bool`

Section: 7.3.13 [conv.fctptr] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2015-05-28

According to 7.3.13 [conv.fctptr] paragraph 1,

> For direct-initialization (9.3 [dcl.init]), a prvalue of type `std::nullptr_t` can be converted to a prvalue of type `bool`; the resulting value is `false`.

The mention of direct-initialization in this context (added by issue 1423) seems odd; standard conversions are on a level below initialization. Should this wording be moved to 9.3 [dcl.init], perhaps as a bullet in paragraph 1?

(See also issue 1781.)

Proposed resolution (June, 2018):

This issue is resolved by the resolution of issue 1781.

2373. Incorrect handling of static member function templates in partial ordering

Section: 12.6.6.2 [temp.func.order] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2018-02-14

According to 12.6.6.2 [temp.func.order] paragraph 3,

> ...If only one of the function templates *M* is a non-static member of some class *A*, *M* is considered to have a new first parameter inserted in its function parameter list. Given *cv* as the cv-qualifiers of *M* (if any), the new parameter is of type “rvalue reference to *cv A*” if the optional *ref-qualifier* of *M* is && or if *M* has no *ref-qualifier* and the first parameter of the other template has rvalue reference type. Otherwise, the new parameter is of type “lvalue reference to *cv A*”. [*Note:* This allows a non-static member to be ordered with respect to a non-member function and for the results to be equivalent to the ordering of two equivalent non-members. —*end note*]

This gives the wrong answer for an example like:

```
struct Foo {
  template <typename T>
  static T* f(Foo*) { return nullptr; }

  template <typename T, typename A1>
  T* f(const A1&) { return nullptr; }
};

int main() {
  Foo x;
  x.f<int>(&x);
}
```

Presumably this should say something like,

> ...If only one of the function templates *M* is a member function, and that function is a non-static member...

Proposed resolution (October, 2018):

Change 12.6.6.2 [temp.func.order] paragraph 3 as follows:

> ...If only one of the function templates *M* is a **member function, and that function is a** non-static member of some class *A*, *M* is considered to have a new first parameter inserted in its function parameter list. Given *cv* as the cv-qualifiers of *M* (if any), the new parameter is of type “rvalue reference to *cv A*” if the optional *ref-qualifier* of *M* is && or if *M* has no *ref-qualifier* and the first parameter of the other template has rvalue reference type. Otherwise, the new parameter is of type “lvalue reference to *cv A*”. [*Note:* This allows a non-static member to be ordered with respect to a non-member function and for the results to be equivalent to the ordering of two equivalent non-members. —*end note*]